

Where to Go From Here

That was a very brief introduction and we only covered topics, not services, actions, parameters, or the many other facets of ROS. Luckily, the online [ROS tutorials](#) are an excellent resource for learning about the rest of ROS. We specifically recommend the [Beginner: Client Libraries](#) collection as a natural next step after reading this chapter.

Regarding the Shortcuts

In this section we have presented the simplest, shortest example ROS programs that we could come up with. Such programs are easy to understand and learn from, as they do not have unnecessary structure or decoration. But in exchange such programs are not easily extensible, composable, or maintainable.

The techniques that we used in the example code in this section are useful for prototyping and experimentation (an important aspect of any good robotics project!), but we do not recommend them for serious work. As you go through the [ROS tutorials](#) and start reading existing ROS code, you will learn about a number of concepts, patterns, and conventions, such as:

- organizing your code into *packages*
- organizing your packages into a *workspace*
- managing *dependencies* among packages
- using the `colcon` tool to build code in multiple packages in dependency order
- using the `ament` module in your `CMakeLists.txt` files
- structuring your code to allow run-time control of how nodes maps to processes
- using the client libraries' console-logging routines for output to screen and elsewhere

These techniques will serve you well when you start building your own ROS applications, especially when you want to share your code with others, whether on your team or out in the world.

Found a bug? [Edit this page on GitHub](#).

Traffic Editor

This section describes the traffic-editor GUI and simulation tools.

Introduction and Objectives

Traffic management of heterogeneous robot fleets is non-trivial. One of the challenges with coordinated management arises from varying semantics in information models used across fleets. Representations of waypoints, lanes, charging/docking stations, restricted zones, infrastructure systems such as doors & lifts, among others, are subject to vendor's discretion. However, standardized conventions that convey the capabilities and intentions of fleets in a shared facility are quintessential for planning. Multi-agent participants in other modes of transportation such as roadways collectively adhere to a set of rules and conventions which minimize chaos. More importantly, they allow for a new participant to readily integrate into the system by following the prescribed rules. Existing agents can accommodate the new participant as its behavior is apparent.

Traffic conventions for multi-robot systems do not exist. The objective of the `traffic_editor` is to fill this gap by expressing the intentions of various fleets in a standardized, vendor neutral manner through a graphical interface. Collated traffic information from different fleets can then be exported for planning and control. A secondary objective and benefit of the `traffic_editor` is to facilitate generation of 3D simulation worlds which accurately reflect physical environments.

Overview

The `traffic_editor` [repository](#) is home to the `traffic_editor` GUI and tools to auto-generate simulation worlds from GUI output. The GUI is an easy-to-use interface which can create and annotate 2D floor plans with robot traffic along with building infrastructure information. Often times, there are existing floor plans of the environment, such as architectural drawings, which simplify the task and provide a "reference" coordinate system for vendor-specific maps. For such cases, `traffic_editor` can import these types of "background images" to serve as a canvas upon which to draw the intended robot traffic maps, and to make it easy to trace the important wall segments required for simulation.

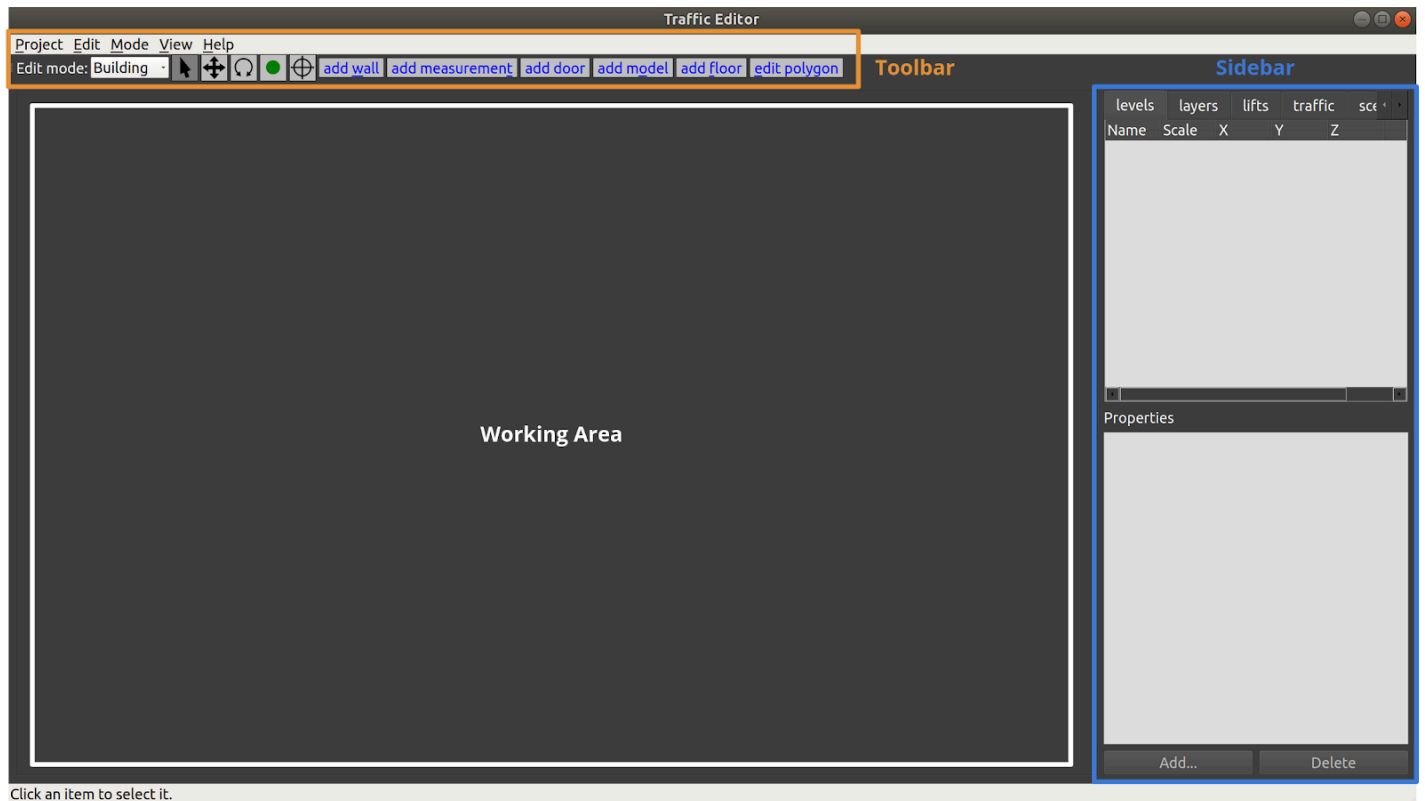
The `traffic_editor` GUI projects are stored as `yaml` files with `.building.yaml` file extensions. Although the typical workflow uses the GUI and does not require hand-editing the

yaml files directly, we have used a yaml file format to make it easy to parse using custom scripting if needed. Each .building.yaml file includes several attributes for each level in the site as annotated by the user. An empty .building.yaml file appears below. The GUI tries to make it easy to add and update content to these file.

```
levels:
  L1:
    doors:
      - []
    drawing:
      filename:
    fiducials:
    elevation: 0
    flattened_x_offset: 0
    flattened_y_offset: 0
    floors:
      - parameters: {}
        vertices: []
    lanes:
      - []
    layers:
      {}
    measurements:
      - []
    models:
      - {}
    vertices:
      {}
    walls:
      {}
  lifts:
    {}
name: building
```

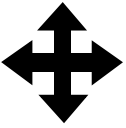
GUI Layout

The layout of the traffic_editor includes a Toolbar ,a Working Area and a Sidebar as seen in the figure below:



The toolbar contains a variety of tools to support actions such as setting the scale of the drawing, aligning levels for multi-level scenarios, adding virtual models to simulated environments, adding robot traffic lanes, simulated flooring, and so on.

As usual in a modern GUI, the top Toolbar contains a variety of tools to interact with items in the main Working Area. This document will introduce and explain the tools as an example project is created. However, the first three tools in the toolbar are commonly found in 2D drawing tools, and should behave as expected:

Icon	Name	Shortcut	Function
	Select	Esc	Select an entity in the Working Area
	Move	m	Move an entity in the Working Area
	Rotate	r	Rotate an entity in the Working Area

The **Working Area** is where the levels, along with their annotations, are rendered. The user is able to zoom via the mouse scroll wheel, and pan the view by pressing the scroll wheel and moving the mouse cursor.

The `sidebar` on the right side of the window contains multiple tabs with various functionalities:

- **levels:** to add a new level to the building. This can be done from scratch or by importing a floor plan image file.
- **layers:** to overlap other images such as lidar maps over the level
- **lifts:** to configure and add lifts to the building
- **traffic:** to select which "navigation graph" is currently being edited, and toggle which graph(s) are being rendered.

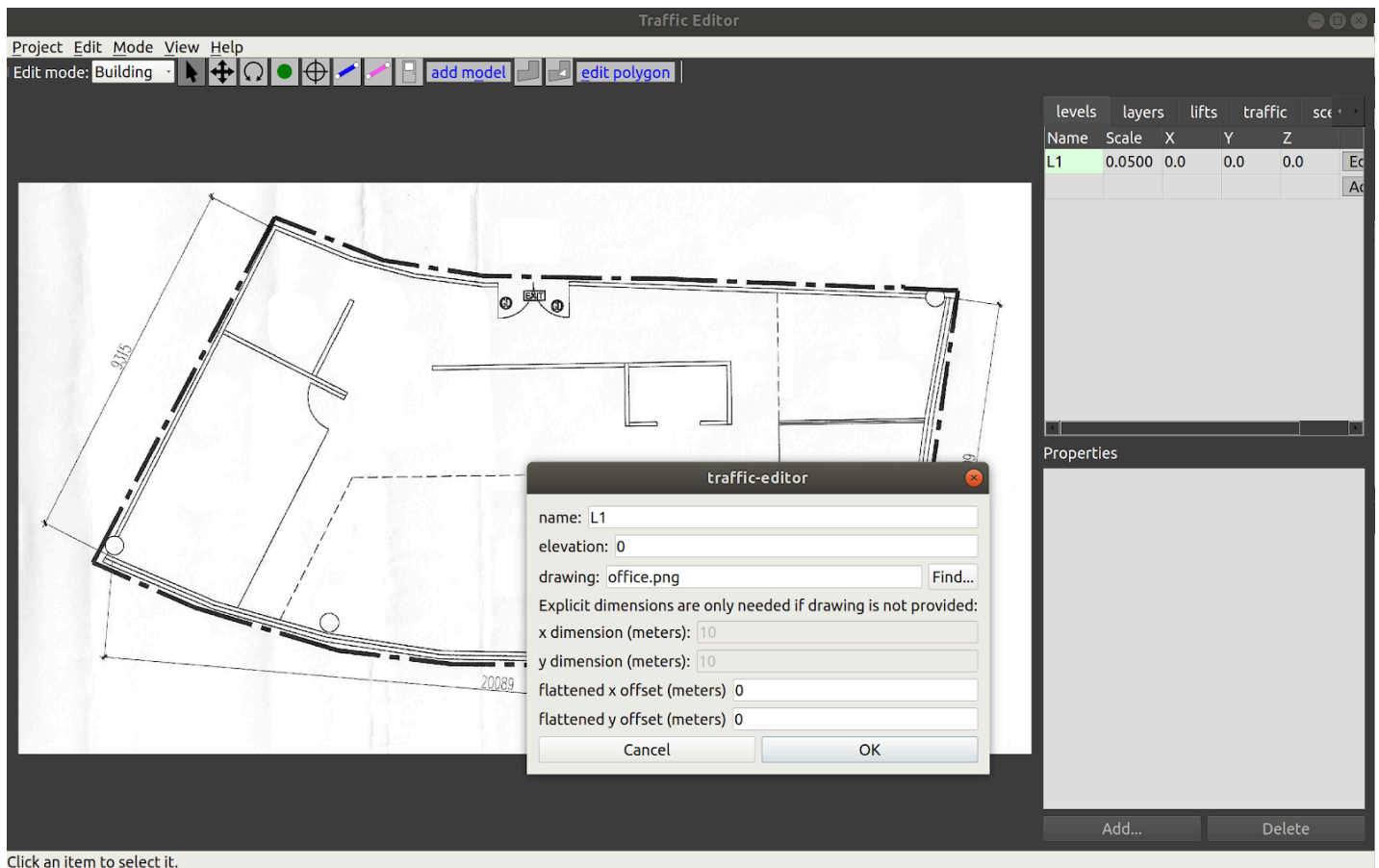
Annotation Guide

This section walks through the process of annotating facilities while highlighting the capabilities of the `traffic_editor` GUI.

To create a new traffic editor `Building` file, launch the traffic editor from a terminal window (first sourcing the workspace if `traffic-editor` is built from source). Then, click `Building -> New...` and choose a location and filename for your `.building.yaml` file.

Adding a level

A new level in the building can be added by clicking the `Add` button in the `levels` tab of the `sidebar`. The operation will open a dialog box where the `name`, `elevation` (in meters) and path to a 2D `drawing` file (`.png`) can be specified. In most use cases, the floor plan for the level is used as the drawing. If unspecified, the user may explicitly enter dimensions of the level in the fields provided.



In the figure above, a new level `L1` at `0m` elevation and a floor plan have been added as reflected in the `levels` tab. A default scale `1px = 5cm` is applied. The actual scale can be set by adding a measurement. Any offsets applied to align levels will be reflected in the `x` and `y` columns. Saving the project will update the `tutorial.building.yaml` files as seen below:

```
levels:
  L1:
    drawing:
      filename: office.png
    elevation: 0
    flattened_x_offset: 0
    flattened_y_offset: 0
    layers:
      {}
lifts:
  {}
name: building
```

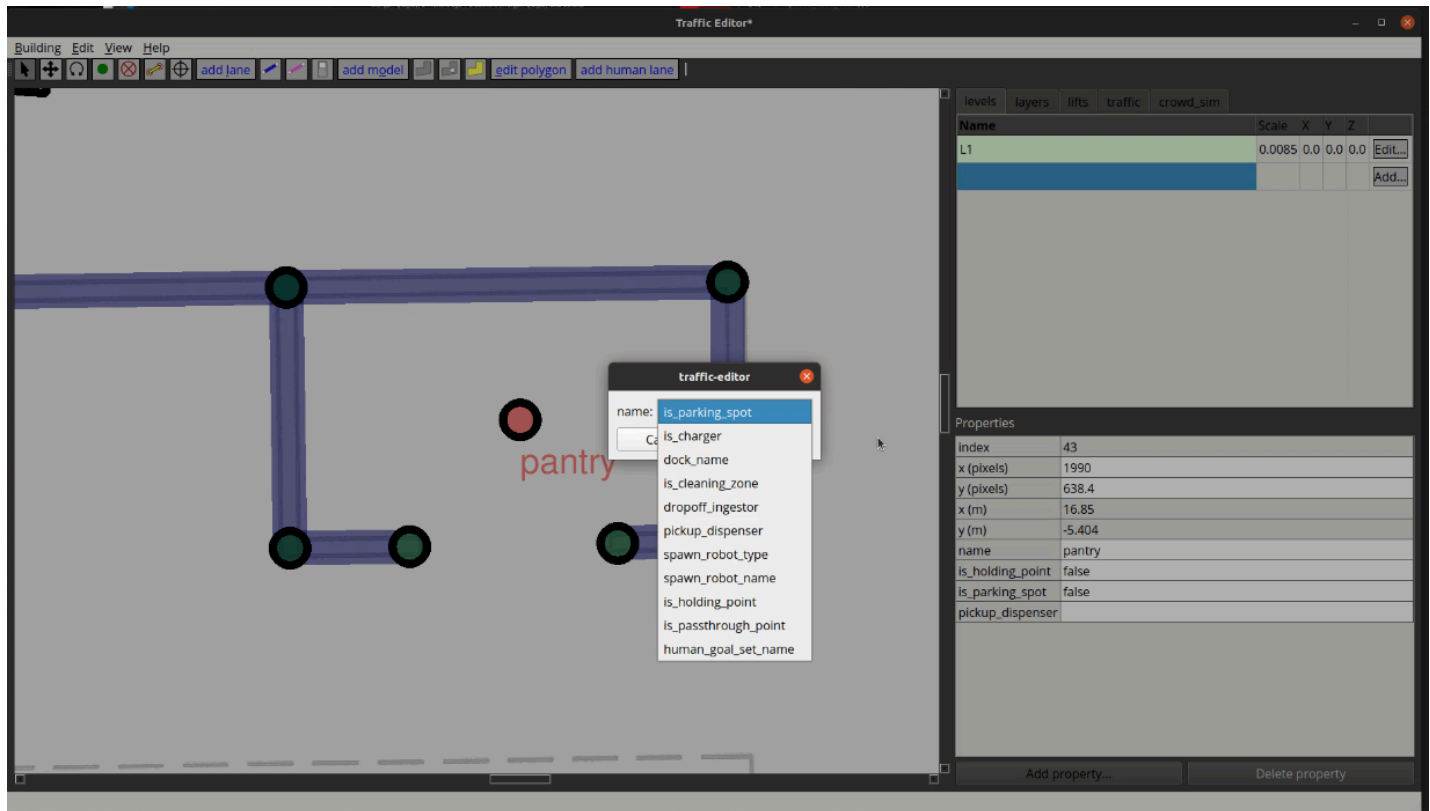
Adding a vertex

Icon	Shortcut
	v

A vertex is a fundamental component of multiple annotations. Walls, measurements, doors, floor polygons and traffic lanes are created from two or more vertices. To create a vertex, click on the vertex icon in the `Toolbar` and then click anywhere on the canvas. The default attributes of a vertex are its coordinates along with an empty name field. Additional attributes may be added by first selecting the vertex (which will turn it red), and then clicking the `Add` button in the figure. Short descriptions of these are presented below:

- **is_holding_point:** if true and if the waypoint is part of a traffic lane, the `rmf_fleet_adapter` will treat this as a *holding point* during path planning, i.e., the robot is allowed to wait at this waypoint for an indefinite period of time.
- **is_parking_spot:** robot's parking spot. [Definition](#)
- **is_passthrough_point:** waypoint which the robot shouldnt stop. [Definition](#)
- **is_charger:** if true and if the waypoint is part of a traffic lane, the `rmf_fleet_adapter` will treat this as a charging station.
- **is_cleaning_zone** indicate if current waypoint is a cleaning zone, specifically for `Clean Task`.
- **dock_name:** if specified and if the waypoint is part of a traffic lane, the `rmf_fleet_adapter` will issue an `rmf_fleet_msgs::ModeRequest` message with `MODE_DOCKING` and `task_id` equal to the specified name to the robot as it approaches this waypoint. This is used when the robot is executing their custom docking sequence (or custom travel path).
- **spawn_robot_type:** the name of the robot model to spawn at this waypoint in simulation. The value must match the model's folder name in the assets repository. More details on the robot model and plugin required for simulation can be found in [Simulation](#)
- **spawn_robot_name:** a unique identifier for the robot spawned at this waypoint. The `rmf_fleet_msgs::RobotState` message published by this robot will have `name` field equal to this value.
- **pickup_dispenser** name of the dispenser workcell for `Delivery Task`, typically is the name of the model. See the [Workcell section] (<https://osrf.github.io/ros2multirobotbook/simulation.html#workcells>) of the Simulation Chapter for more details.
- **dropoff_ingestor** name of the ingestor workcell for `Delivery Task`, typically is the name of the model. See the [Workcell section] (<https://osrf.github.io/ros2multirobotbook/simulation.html#workcells>) of the Simulation Chapter for more details.

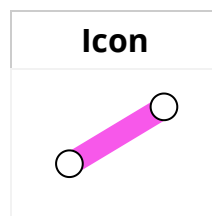
- **human_goal_set_name** The `goal_sets.set_area` name, used by crowd simulation. For more info about `crowd_sim`, please see the [Crowdsim section] (<https://osrf.github.io/ros2multirobotbook/simulation.html#crowdsim>) of the Simulation Chapter for more details.



Each vertex is stored in the `tutorial.building.yaml` file as a list of x-coordinate, y-coordinate, elevation, vertex_name and a set of additional parameters.

```
vertices:
  - [1364.76, 1336.717, 0, magni1_charger, {is_charger: [4, true],
is_parking_spot: [4, true], spawn_robot_name: [1, magni1], spawn_robot_type: [1,
Magni]]}]
```

Adding a measurement

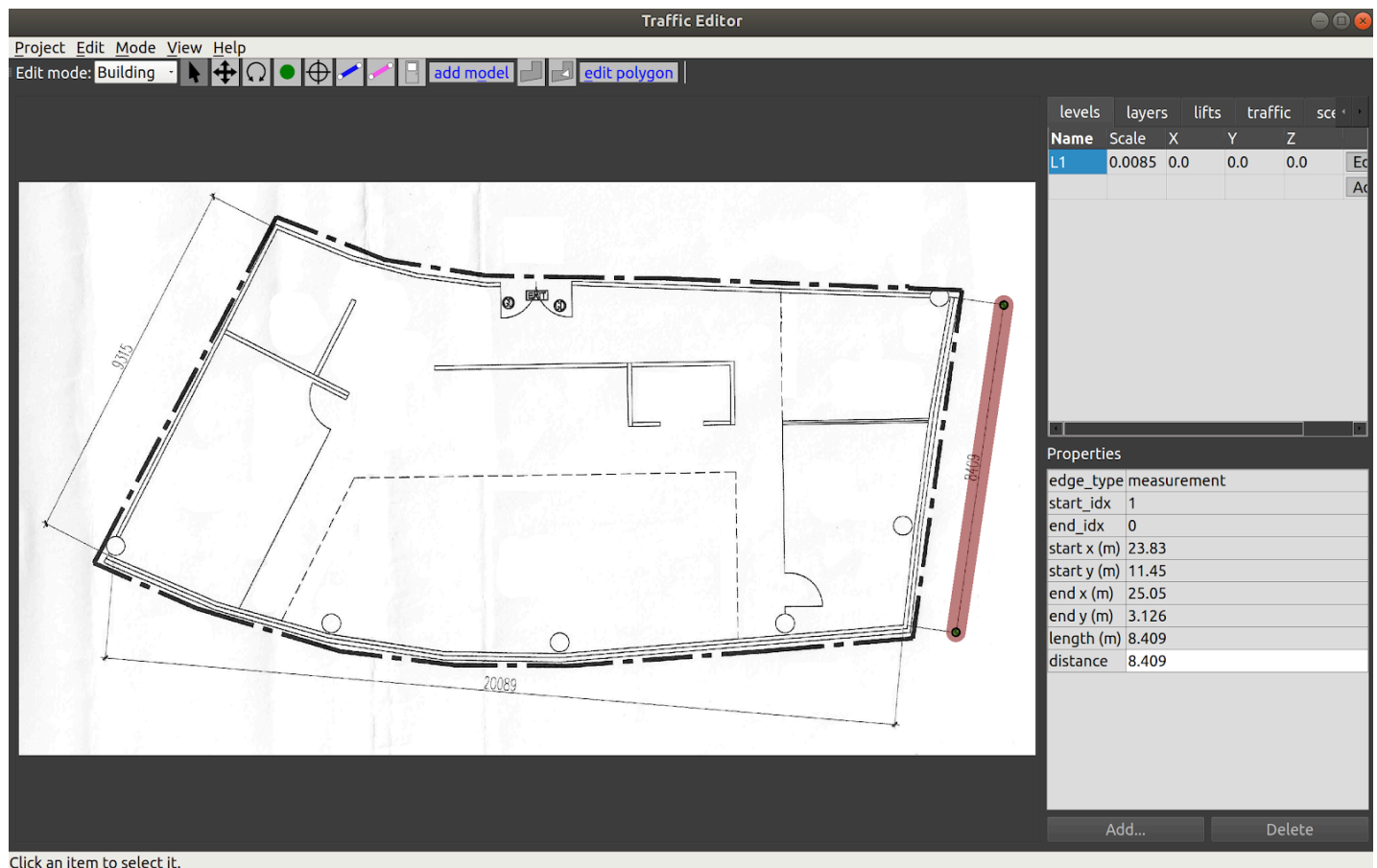


Adding a measurement sets the scale of the imported 2D drawing, which is essential for planning and simulation accuracy. Scalebars or reference dimensions in the floor plan aid with the process. Often, you can draw the measurement line directly on top of a reference scale bar

in a drawing. With the editor in *Building* mode, select the *Add Measurement* tool and click on two points with known dimensions. A pink line is rendered on the map with two vertices at its ends at the selected points.

Note: A measurement line may be drawn by clicking on existing vertices. In this scenario, no additional vertices are created at its ends.

Selecting the line populates various parameters in the Properties window of the sidebar. Setting the `distance` parameter to the physical distance between the points (in meters) will then update the `scale` for the level. Currently, you must save the project and restart `traffic-editor` to see the changes reflected (todo: fix this...).



The above process adds two `vertices` and a `measurement` field to the `tutorial.building.yaml` file as seen below. For the measurement field, the first two elements represent the indices of vertices representing the ends of the line. The `distance` value is stored in a sub-list of parameters.

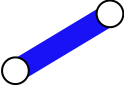
```

levels:
  L1:
    drawing:
      filename: office.png
      elevation: 0
      flattened_x_offset: 0
      flattened_y_offset: 0
      layers:
        {}
    measurements:
      - [1, 0, {distance: [3, 8.409]}]
    vertices:
      - [2951.728, 368.353, 0, ""]
      - [2808.142, 1348.9, 0, ""]

lifts:
  {}
name: building

```

Adding a wall

Icon	Shortcut
	w

To annotate walls in the map, select the *Add Wall* icon from the `Toolbar` and click on consecutive vertices that represent the corners of the wall. The process of adding wall segments is continuous, and can be exited by pressing the `Esc` key. Blue lines between vertices are rendered on the map which represent the drawn walls. If the corner vertices are not present, they will automatically be created when using this tool. Meshes of the annotated walls are automatically generated during 3D world generation using `building_map_generator`. By default, the walls are of thickness of 10cm and height 2.5m. The `wall_height` and `wall_thickness` attributes may be modified [in the source code](#).

Wall texture options are available [here](#) in the source code.